UNIVERSITY OF ATHENS
DEPARTMENT OF INFORMATICS AND TELECOMMUNICATIONS

Deep Learning for NLP

Student name: *Antigoni Stefanou*
sdi: sdi2000281

Course: *Artificial Intelligence II (M138, M226, M262, M325)*
Semester: *Spring Semester 2025*

Contents

1. Abstract

In this assignment, the task is to categorize tweets, depending on whether the sentiment behind them is positive or negative. For this purpose, the data will be converted to GloVe word embeddings and then will be given to a feed forward neural network. The relevant notebook can be found here:
https://www.kaggle.com/code/antigonistefanou/sdi2000281-hw2

2. Data processing

2.1. Vectorization

In order for the model to have a nuanced understanding of language and the semantic relationship between words, the input will be converted to word embeddings. Specifically, GloVe's Twitter word vector will be used, as it has been pretrained on similar data. This way the mean vector for each tweet can be calculated and then be given to the model for training.

2.2. Pre-processing

One of the most important tasks in this assignment is the pre-processing, the goal of which is to remove the noise often found in tweets, in order to improve the model's performance. Keeping in mind that the data will be represented by GloVe vectors, it is important to transform the input into words that exist within them. Glove's website contains a Ruby script for preprocessing Twitter data.[4] By looking into it, it is apparent that special tags (contained in the "<" and ">" characters) are used to signal information about a tweet. This should be taken into account as it can impact the model's understanding of sentiment. In the end, the following methods were used to clean the datasets.

1. Replace all emojis with their CLDR short name, aka a word or small phrase that describes them. This is done with the help of the demojize function found in the emoji module.
2. Remove all HTML character entities. These entities start with an ampersand and end with a semicolon (ex. <), so they can be found using RegEx.
3. Replace all phone numbers with the word "<number>". This is also important for privacy reasons.
4. Find all links starting with "http(s)://" or "www." and replace them with the word "<url>".
5. Find all emails and replace them with the word "EMAIL". This is also important for privacy reasons.
6. Find all Twitter handles and replace them with the word "<user>". This is also important for privacy reasons.
7. Replace all hashtag symbols with the word "<hashtag>".

8. Remove all punctuation marks and special characters, except "<" and ">" that denote a special tag such as the ones mentioned above.

9. Remove non-ASCII characters.

10. Search for all instances where a letter appears more than twice in a row and replace it with just two characters. This way, words such as "good" and "gooooooood" will be recognized as the same word. An unintended side effect here is that the original word might have only contained one. To combat this, the original Ruby script provided the "<elong>" tag. However, in practice, this tag offered negligible improvement and was thus omitted.

11. Convert all letters to lowercase, since GloVe is case sensitive and pretrained on lowercase data. This way, there will be no distinction between words such as "Hello" and "hello".

Not all of the methods found in the GloVe preprocessing script were utilized, as some had adverse effects in the model's accuracy.

3. Algorithms and Experiments

3.1. Algorithms

First and foremost, the functions to be used in the training and evaluation of the model must be defined. The training function includes

- a standard backpropagation step, during which the training loss is calculated,
- an early stopping implementation, that stops the training if the validation loss doesn't drop under a specified margin(delta) within a limited number of iterations(patience),
- a scheduler, that updates the learning rate dynamically, thus preventing the model from getting stuck on local minima or other suboptimal points.

The evaluation function predicts the validation dataset's labels using the trained model and then calculates its loss.

The framework used for the above routines is Torch's Python implementation, PyTorch[6] and specifically its neural network library, torch.nn.

3.2. Experiments

To effectively tackle this problem, a baseline is needed. Therefore the most simple binary classifier is created.

```
class BaseNet(nn.Module):
    def __init__(self, D_in, H, D_out):
        super(BaseNet, self).__init__()
        self.linear_stack = nn.Sequential(
            nn.Linear(D_in, H),
            nn.ReLU(),
            nn.Linear(H, D_out))
```

```
    def forward(self, data):
        output = self.linear_stack(data)
        return output
```

Listing 1: BaseNet

The model, after being trained with this perceptron, yields a loss of 0.44 and an accuracy score of 0.78 on the validation set. These are the scores to be improved upon.

The next step is configuring a more complex architecture that will be more sensitive to patterns in the input data. There are several architectural choices to consider, namely the number of hidden layers and nodes, the activation function and the inclusion/exclusion of batch normalization and dropout. Initially, Optuna (see 3.3) was used to adjust the number of hidden layers/nodes and batch normalization features. Doing so, however, substantially increased the search space during hyperparameter tuning while also offering little improvement in the model's performance. Therefore, these parameters were manually chosen with trial and error.

Because the task is a binary classification problem, not a lot of hidden layers are needed. In fact, two Linear transformation layers proved most effective. An activation function is necessary to add non-linearity to the model and ReLU is a simple and efficient way to do so. Batch normalization was also included, as it kept the model's training steady and effective, even though the input dataloaders have a relatively high batch size of 128. Finally, Dropout was included in the neural network, as a method to help prevent overfitting. The dropout factor was determined during hyperparameter tuning. Below is the final network.

```
class Net(nn.Module):
    def __init__(self, D_in, D_out):
        super(Net, self).__init__()
        self.linear_stack = nn.Sequential(
            nn.Linear(D_in, 768),
            nn.BatchNorm1d(768),
            nn.ReLU(inplace=True),
            nn.Dropout(dropout_factor),
            nn.Linear(768, 256),
            nn.BatchNorm1d(256),
            nn.ReLU(inplace=True),
            nn.Dropout(dropout_factor),
            nn.Linear(256, D_out),
        )

    def forward(self, data):
        output = self.linear_stack(data)
        return output
```

Listing 2: A More Complex Network

Further experimentation came with the implementation of early stopping and scheduler, as mentioned in section 3.1. Early stopping is a way for the training to stop when a specified metric doesn't marginally improve over a set number of iterations. In this case, the metric is the validation set's loss. The margin of improvement, aka the delta, as well as the stopper's patience will be determined during hyperparameter tuning. The scheduler is a method used to avoid setting a static learning rate for the model, making it a very powerful tool in effectively training it. This is because a higher learning rate might miss the most optimal point and, conversely, a lower learning rate may be too computationally intense and get stuck in local minima. The scheduler used is

Torch's ReduceLROnPlateau, so that it will monitor the validation loss. Its factor, by which the learning rate will be reduced, and the optimizer used will be determined during the optimization step. With these two inclusions, the problem of overfitting is overcome.

The final step before training the model is hyperparameter tuning. This was achieved with the Optuna framework and is analyzed in depth in Section 3.3. The only hyperparameter that was decided independently was the loss function. BCEWithLogitsLoss was judged to be the most appropriate function for this task. This is because it combines BCE loss, an already reliable choice in binary classification problems, and a Sigmoid layer in one class, while also being more numerically stable than using them sequentially. [5]

After the network's architecture has been established and Optuna has applied the necessary optimization, the focus shifts to ensemble predictions. The model is run 10 times, each with a different seed, and the probability for each prediction is averaged, before being rounded to 0/1. By training multiple models, the ensemble algorithm can combine the outcomes and reach a more accurate final prediction. This process allows the model to become more robust by eliminating uncorrelated errors, while reinforcing correct predictions.

3.3. Hyper-parameter tuning and Optimization techniques

In order to achieve optimal results, the model's hyperparameters need to be tuned. This was done with the help of the Optuna optimization framework. More specifically a study was created that, through 300 trials, would suggest the optimal values for the following

- optimizer, choosing between Adam and AdamW. Adam is a broadly used optimizer, while also being straightforward to implement, computationally efficient, with little memory requirements.[2] AdamW is a version of Adam that applies weight decay directly during the parameter update, leading to substantial improvement in Adam's generalization performance.[3]
- initial learning rate, from the range [1e-7, 1e-4]
- scheduler factor, from the range [0.3, 0.7]
- early stopping delta, from the range [1e-6, 1e-4]
- early stopping patience, from the range [1, 6]
- dropout factor, from the range [0.15, 0.3]

3.4. Evaluation

The following metrics will be used to evaluate the models.

3.4.1. Accuracy.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Accuracy is a metric that measures how often a model correctly predicts the outcome, in this case the tweet's sentiment. A high accuracy score means that the model can correctly categorize the data. However, it is not enough on it's own, as a very high accuracy is also a byproduct of overfitting.

3.4.2. Per-Class Metrics Barplot. A barplot that showcases the accuracy, precision, recall and F1 score for each of the two classes. With it, it becomes apparent if the model is biased towards one of the classes.

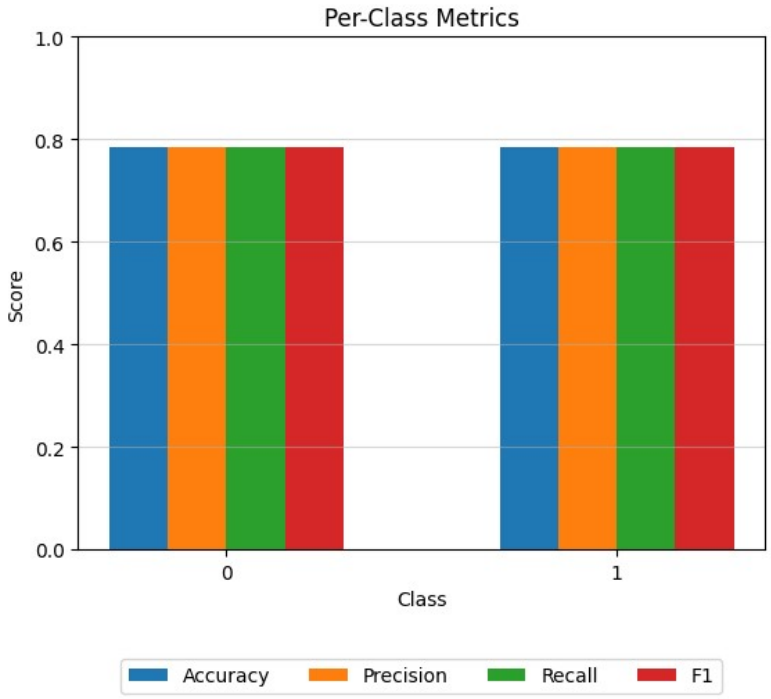

Figure 1: Base Model Per Class Metrics

3.4.3. Confusion matrix. The confusion matrix is a 2x2 table with the number of correct predictions (true positives and true negatives) and incorrect predictions (false positives and false negatives) for both classes. It is a very important metric, since it clearly presents the model's weak points.

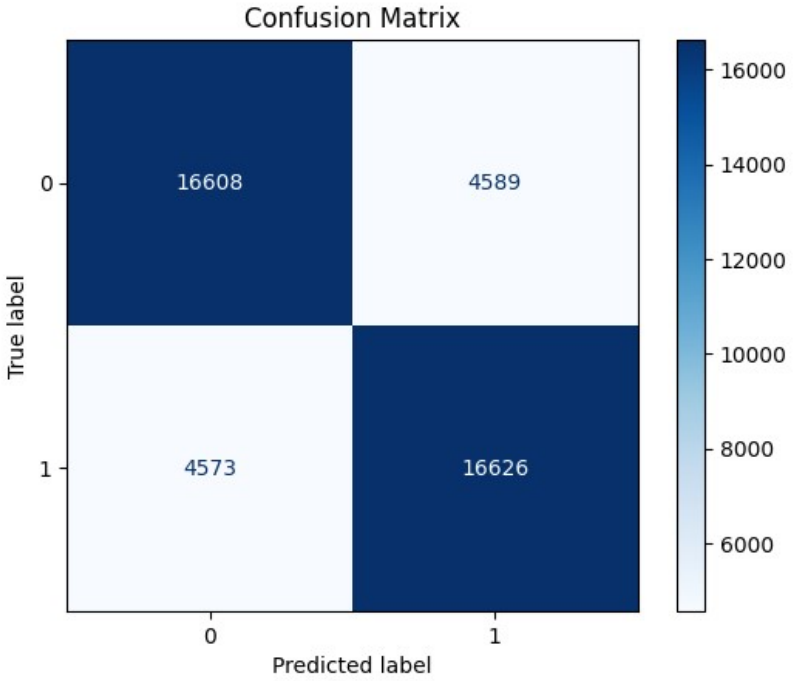

Figure 2: Base Model Confusion Matrix

3.4.4. ROC curve. The ROC curve shows the performance of a binary classifier. It is the plot of the true positive rate (TPR) against the false positive rate (FPR). The area under the ROC curve (AUC) is a good way to summarize in a single number the diagnostic ability of the classifier.[1] The more expansive the area, the better the classifier.

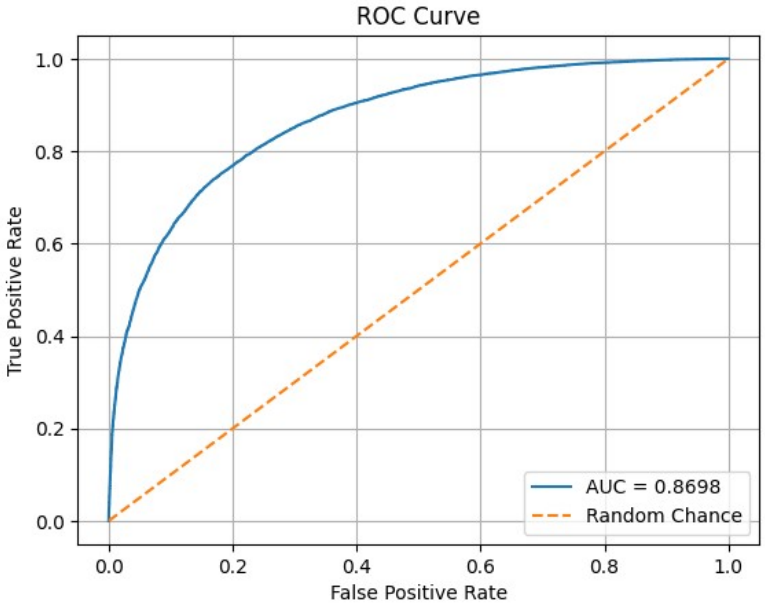

Figure 3: Base Model ROC Curve

3.4.5. Training and Validation Loss curve. A curve that illustrates both the training and validations losses as the epochs progress. Ideally, both losses decrease overtime, signaling that the model is fitting the training data and can generalize well to new (validation) data.

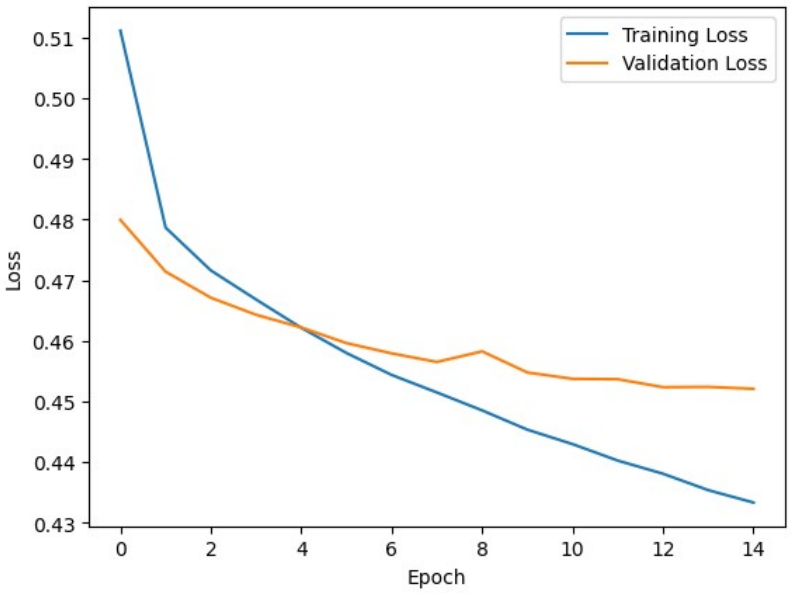

Figure 4: Base Model Loss Curve

3.4.6. Distribution of Probabilities. This figure shows the probabilities generated by the model before they are rounded to 0/1. Therefore, it presents well how certain the model is with its predictions in general, as well as its confidence for each class.

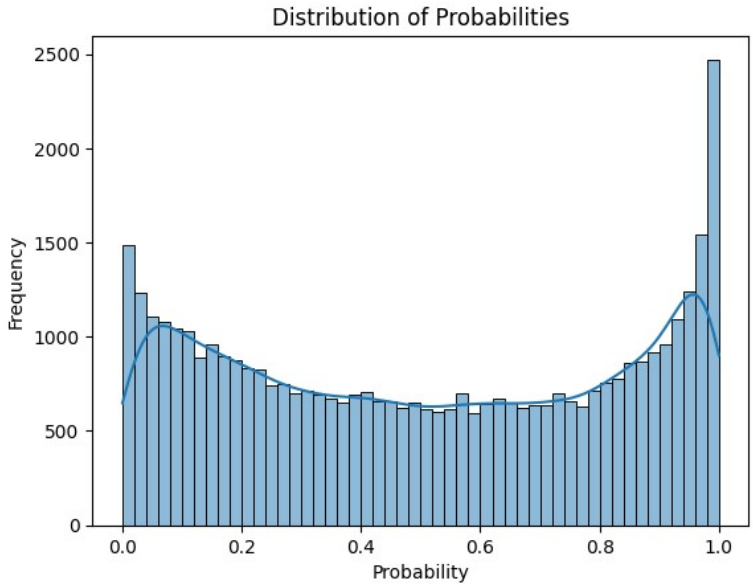

Figure 5: Base Model Distribution of Probabilities

4. Results and Overall Analysis

4.1. Results Analysis

In the end, the best model can correctly predict a tweet's sentiment with around 79% accuracy, which is a relatively high score. The precision, recall, F1 and ROC AUC scores are also similarly high (as seen in Figures 6 and 8), which, in combination with the early stopping and scheduling methods that have been implemented, can be interpreted as a steep reduction of overfitting in the model.

In theory, the model could be improved further by being trained on both the training and validation sets after establishing the architecture and hyperparameters. However, this was not possible with the inclusion of early stopping and the ReduceLROnPlateau scheduler, as they depend on the existence of a validation loss. It is possible that the inclusion of the extra tweets, given that the quality of the data and the class balance was the same, would improve the predictive capabilities of the model by a small margin and reduce underfitting.

Upon studying Figure 9, it is apparent that the predicted probabilities are not concentrated exclusively near the decision boundaries, but near intermediate values as well. This shows a lack of confidence from the model. While it could be argued that this indicates a need for further development, it is important to consider the nature of the input as well. Human expression, and by extension Tweets, is not always inherently positive or negative in sentiment. Ergo, the observed uncertainty might be a reflection of the nuance and ambiguity of language, rather than the model's limitations.

4.1.1. Best trial. The best model was achieved with the following neural network architecture

```
class Net(nn.Module):
    def __init__(self, D_in, D_out):
        super(Net, self).__init__()
        self.linear_stack = nn.Sequential(
            nn.Linear(D_in, 768),
            nn.BatchNorm1d(768),
            nn.ReLU(inplace=True),
            nn.Dropout(0.28),
            nn.Linear(768, 256),
            nn.BatchNorm1d(256),
            nn.ReLU(inplace=True),
            nn.Dropout(0.28),
            nn.Linear(256, D_out),
        )

    def forward(self, data):
        output = self.linear_stack(data)
        return output
```

Listing 3: Final Network

The best hyperparameters are the following:

Optimizer	
name	AdamW
lr	7.329999999999999e-05
Scheduler	
name	ReduceLROnPlateau
factor	0.32
patience	2
min_lr	1e-7
Early Stopping	
patience	4
delta	4.599999999999999e-05

It achieved the following scores on the validation dataset:

Accuracy	0.7945
ROC AUC Score	0.8797
F1 Score	0.7937

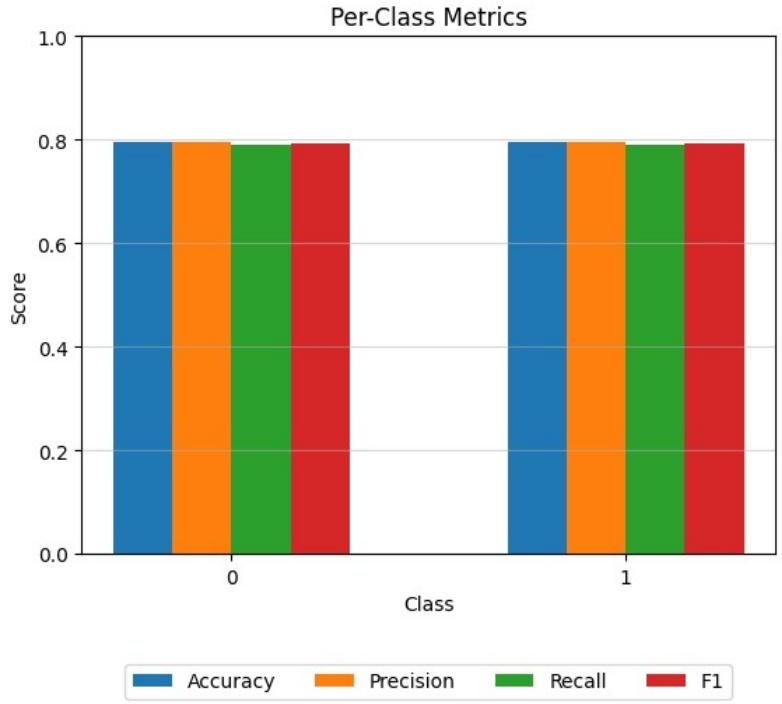

Figure 6: Best Model Per Class Metrics

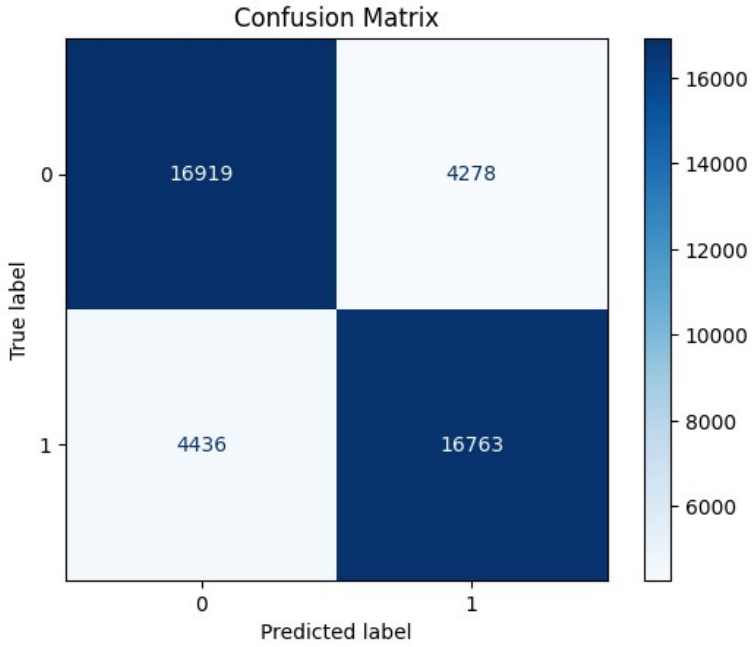

Figure 7: Best Model Confusion Matrix

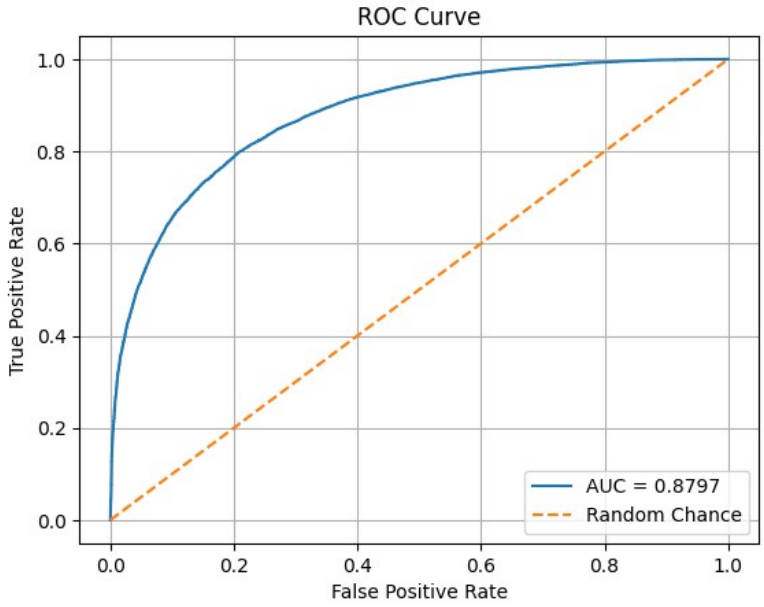

Figure 8: Best Model ROC Curve

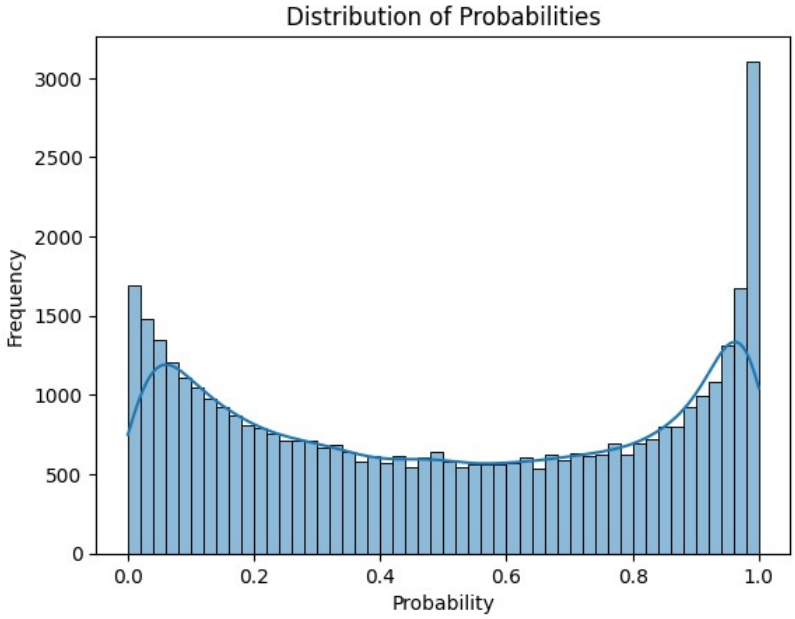

Figure 9: Best Model Distribution of Probabilities

The test dataset's accuracy score is 0.79277.

4.2. Comparison with the first project

Comparatively to the first project, the model performs similarly, if slightly worse, and required a more sophisticated approach in its optimization. This is to be expected, as neural networks provide a plethora of options regarding their architecture, hyperparameters, as well as regularization and optimization methods. While this makes their development more time consuming, it also gives the opportunity to create a heavily personalized model for the task at hand.

5. Bibliography

References

[1] Andrew P. Bradley. The use of the area under the roc curve in the evaluation of machine learning algorithms. *Pattern Recognition*, 30(7):1145–1159, 1997.

[2] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization, 2017.

[3] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization, 2019.

[4] Romain Paulus and Jeffrey Pennington. Script for preprocessing tweets, 2023. Accessed: 2025-05-04.

[5] PyTorch Team. BCEWithLogitsLoss, 2024. Accessed: 2025-05-04.

[6] PyTorch Team. PyTorch, 2024. Accessed: 2025-05-04.